

Operators and Control Statements in Java

Lecture 6 and 7

Instructor:

Shovon Mandal (SM), Lecturer

Department of Computer Science and Engineering (CSE),
Northern University & Business Technology Khulna (NUBTK)

Operators in Java

Lecture 6

Operator

- An operator is a symbol that operates on one or more arguments to produce a result.
- Java provides a rich set of operators to manipulate variables.

Operands

- An operands are the values on which the operators act upon.
- An operand can be:
 - **A numeric variable** - integer, floating point or character
 - **Any primitive type variable** - numeric and Boolean
 - **Reference variable** to an object
 - **A literal** - numeric value, Boolean value, or string.
 - **An array element**, "a[2]"
 - **char primitive**, which in numeric operations is treated as an unsigned two-byte integer

Types of Operators

- Assignment Operators
- Increment Decrement Operators
- Arithmetic Operators
- Modulus Operator
- Bitwise Operators
- Relational Operators
- Boolean Logical Operators
- Ternary Operators
- Comma Operators
- Instance of Operators

Assignment Operators

- The assignment statements has the following syntax:

- **<variable> = <expression>**

- Assigning primitive value

```
int a, b;
```

```
a = 2; // 2 is assigned to variable a
```

```
b = 5; // 5 is assigned to variable b
```

- Assigning references

```
Home home1 = new Home(); //new object created
```

```
Home home2 = home1; //assigning the reference of home1 in home2
```

Increment and Decrement operators

++ and --

- The increment and decrement operators add an integer variable by one.
- increment operator:
- two successive plus signs, ++
- decrement operator: --

Common

`a = a + 1;`

`a = a - 1;`

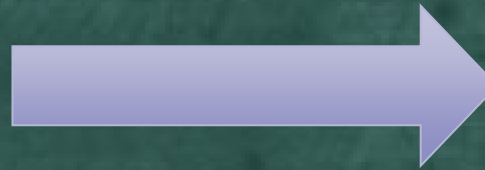
Shorthand

`a++; or ++a;`

`a--; or --a;`

EXAMPLE OF ++ AND -- OPERATORS

```
public class Example {  
    public static void main(String[] args) {  
        int j, p, q, r, s;  
        j = 5;  
        p = ++j; // j = j + 1; p = j;  
        System.out.println("p = " + p);  
        q = j++; // q = j; j = j + 1;  
        System.out.println("q = " + q);  
        System.out.println("j = " + j);  
        r = --j; // j = j - 1; r = j;  
        System.out.println("r = " + r);  
        s = j--; // s = j; j = j - 1;  
        System.out.println("s = " + s);  
    }  
}
```



```
> java example  
p = 6  
q = 6  
j = 7  
r = 6  
s = 6  
>
```

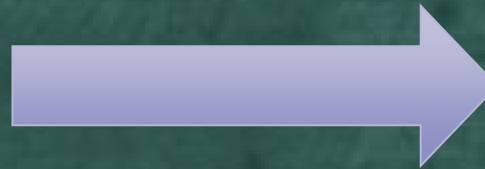

Arithmetic Operators

- The arithmetic operators are used to construct mathematical expressions as in algebra.
- Their operands are of numeric type.

Operator	Result
+	Addition
-	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus
++	Increment
+=	Addition assignment
-=	Subtraction assignment
*=	Multiplication assignment
/=	Division assignment
%=	Modulus assignment
--	Decrement

Example of Arithmetic

```
public class Example {  
    public static void main(String[] args) {  
        int j, k, p, q, r, s, t;  
        j = 5;  
        k = 2;  
        p = j + k;  
        q = j - k;  
        r = j * k;  
        s = j / k;  
        t = j % k;  
        System.out.println("p = " + p);  
        System.out.println("q = " + q);  
        System.out.println("r = " + r);  
        System.out.println("s = " + s);  
        System.out.println("t = " + t);  
    }  
}
```



```
> java Example  
p = 7  
q = 3  
r = 10  
s = 2  
t = 1  
>
```

The Modulus Operator

- The modulus operator, %, returns the remainder of a division operation. It can be applied to floating-point types as well as integer types. The following example program demonstrates the %:

```
// Demonstrate the % operator.
class Modulus {
    public static void main(String args[]) {
        int x = 42;
        double y = 42.25;

        System.out.println("x mod 10 = " + x % 10);
        System.out.println("y mod 10 = " + y % 10);
    }
}
```

When you run this program, you will get the following output:

```
x mod 10 = 2
y mod 10 = 2.25
```

Bitwise Operators

- Java's *bitwise* operators operate on individual bits of integer (int and long) values.
- If an operand is shorter than an int, it is promoted to int before doing the operations.

Operator	Name	Description
$a \& b$	and	1 if both bits are 1.
$a b$	or	1 if either bit is 1.
$a \wedge b$	xor	1 if both bits are different.
$\sim a$	not	Inverts the bits.
$n \ll p$	left shift	Shifts the bits of n left p positions. Zero bits are shifted into the low-order positions.
$n \gg p$	right shift	Shifts the bits of n right p positions. If n is a 2's complement signed number, the sign bit is shifted into the high-order positions.
$n \ggg p$	right shift	Shifts the bits of n right p positions. Zeros are shifted into the high-order positions.

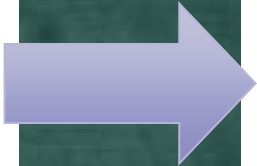
Bitwise Operators Cont.

Operator	Result
~	Bitwise unary NOT
&	Bitwise AND
	Bitwise OR
^	Bitwise exclusive OR
>>	Shift right
>>>	Shift right zero fill
<<	Shift left
&=	Bitwise AND assignment
=	Bitwise OR assignment
^=	Bitwise exclusive OR assignment
>>=	Shift right assignment
>>>=	Shift right zero fill assignment
<<=	Shift left assignment

Bitwise Operators Example 1

```
// Demonstrate the bitwise logical operators.
class BitLogic {
    public static void main(String args[]) {
        String binary[] = {
            "0000", "0001", "0010", "0011", "0100", "0101", "0110", "0111",
            "1000", "1001", "1010", "1011", "1100", "1101", "1110", "1111"
        };
        int a = 3; // 0 + 2 + 1 or 0011 in binary
        int b = 6; // 4 + 2 + 0 or 0110 in binary
        int c = a | b;
        int d = a & b;
        int e = a ^ b;
        int f = (~a & b) | (a & ~b);
        int g = ~a & 0x0f;

        System.out.println("      a = " + binary[a]);
        System.out.println("      b = " + binary[b]);
        System.out.println("      a|b = " + binary[c]);
        System.out.println("      a&b = " + binary[d]);
        System.out.println("      a^b = " + binary[e]);
        System.out.println(" ~a&b|a&~b = " + binary[f]);
        System.out.println("      ~a = " + binary[g]);
    }
}
```



```
      a = 0011
      b = 0110
      a|b = 0111
      a&b = 0010
      a^b = 0101
 ~a&b|a&~b = 0101
      ~a = 1100
```


Bitwise Operators Example 2

```
class Test {  
    public static void main(String args[]) {  
        int a = 60; /* 60 = 0011 1100 */  
        int b = 13; /* 13 = 0000 1101 */  
        int c = 0;  
        c = a & b; /* 12 = 0000 1100 */  
        System.out.println("a & b = " + c );  
        c = a | b; /* 61 = 0011 1101 */  
        System.out.println("a | b = " + c );  
        c = a ^ b; /* 49 = 0011 0001 */  
        System.out.println("a ^ b = " + c );  
        c = ~a; /* -61 = 1100 0011 */
```

```
        System.out.println("~a = " + c );  
        c = a << 2; /* 240 = 1111 0000 */  
        System.out.println("a << 2 = " + c );  
        c = a >> 2; /* 15 = 0000 1111 */  
        System.out.println("a >> 2 = " + c );  
        c = a >>> 2; /* 15 = 0000 1111 */  
        System.out.println("a >>> 2 = " + c );  
    } }
```

Output:

a & b = 12

a | b = 61

a ^ b = 49

~a = -61

a << 2 = 240

a >> 15 a >>> 15

Relational Operators

- A relational operator compares two values and determines the relationship between them.
- For example, != returns true if its two operands are unequal.
- Relational operators are used to test whether two values are equal, whether one value is greater than another, and so forth.

Operator	Description	Operator	Description
==	Checks if the value of two operands are equal or not, if yes then condition becomes true.	<	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.
!=	Checks if the value of two operands are equal or not, if values are not equal then condition becomes true.	>=	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.
>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	<=	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.

Example of Relational Operators

```
public LessThanExample
{
    public static void main(String args[])
    {
        int a = 5; int b = 10;
        if(a < b)
        {
            System.out.println("a is less than b");
        }
    }
}
```

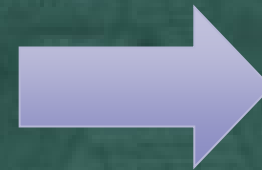
Boolean Logical Operators

- The Boolean logical operators shown here operate only on boolean operands. All of the binary logical operators combine two boolean values to form a resultant Boolean value.

Operator	Result
&	Logical AND
	Logical OR
^	Logical XOR (exclusive OR)
	Short-circuit OR
&&	Short-circuit AND
!	Logical unary NOT
&=	AND assignment
=	OR assignment
^=	XOR assignment
==	Equal to
!=	Not equal to
?:	Ternary if-then-else

Boolean Logical Operators Example

```
// Demonstrate the boolean logical operators.
class BoolLogic {
    public static void main(String args[]) {
        boolean a = true;
        boolean b = false;
        boolean c = a | b;
        boolean d = a & b;
        boolean e = a ^ b;
        boolean f = (!a & b) | (a & !b);
        boolean g = !a;
        System.out.println("a = " + a);
        System.out.println("b = " + b);
        System.out.println("a|b = " + c);
        System.out.println("a&b = " + d);
        System.out.println("a^b = " + e);
        System.out.println("!a&b|a&!b = " + f);
        System.out.println("!a = " + g);
    }
}
```



```
a = true
b = false
a|b = true
a&b = false
a^b = true
!a&b|a&!b = true
!a = false
```

Ternary Operators

- Java has a shorthand way by using **?:** the ternary aka conditional operator for doing ifs that compute a value.
- Unlike the **if statement**, the conditional **operator** is an expression which can be used for

Example of Ternary Operator

// longhand with if:

```
int answer;  
if ( a > b )  
{  
    answer = 1;  
}  
else  
{  
    answer = -1;  
}
```

// can be written more tersely with the ternary operator as:

```
int answer = a > b ? 1 : -1;
```

Comma Operators

- Java has an often look past feature within it's for loop and this is the comma operator.
- Usually when people think about commas in the java language, they think of a way to split up arguments within a function's parameters

Comma Operators Example

```
public class CommaOperator {  
    public static void main(String[] args) {  
        for (int i = 1, j = i + 10; i < 5; i++, j  
= i * 2) {  
            System.out.println("i= " + i + " j=  
" + j);  
        }  
    }  
}
```

//Output:

i= 1 j= 11

i= 2 j= 4

i= 3 j= 6

i= 4 j= 8

Explanation:

First Iteration: $i = 1$, $j = i + 10 = 11$

Prints: $i = 1$ $j = 11$

Then, $i++$ makes $i = 2$, and $j = i * 2 = 4$

Second Iteration: $i = 2$, $j = 4$

Prints: $i = 2$ $j = 4$

Then, $i++$ makes $i = 3$, and $j = i * 2 = 6$

Third Iteration: $i = 3$, $j = 6$

Prints: $i = 3$ $j = 6$

Then, $i++$ makes $i = 4$, and $j = i * 2 = 8$

Fourth Iteration: $i = 4$, $j = 8$

Prints: $i = 4$ $j = 8$

Then, $i++$ makes $i = 5$, which fails the condition $i < 5$, so the loop terminates.

InstanceOf Operators

- This operator is used only for object reference variables. The operator checks whether the object is of a particular type(class type or interface type).
- InstanceOf operator is written as:

(Object reference variable) instanceof (class/interface type)

Example of instanceof Operator

```
class Vehicle {  
    public class Car extends Vehicle  
    {  
  
        public static void main(String args[])  
        {  
  
            Vehicle a = new Car();  
            boolean result = a instanceof Car;  
            System.out.println( result);  
        }  
    }  
}
```

Thank You

08-Nov-25

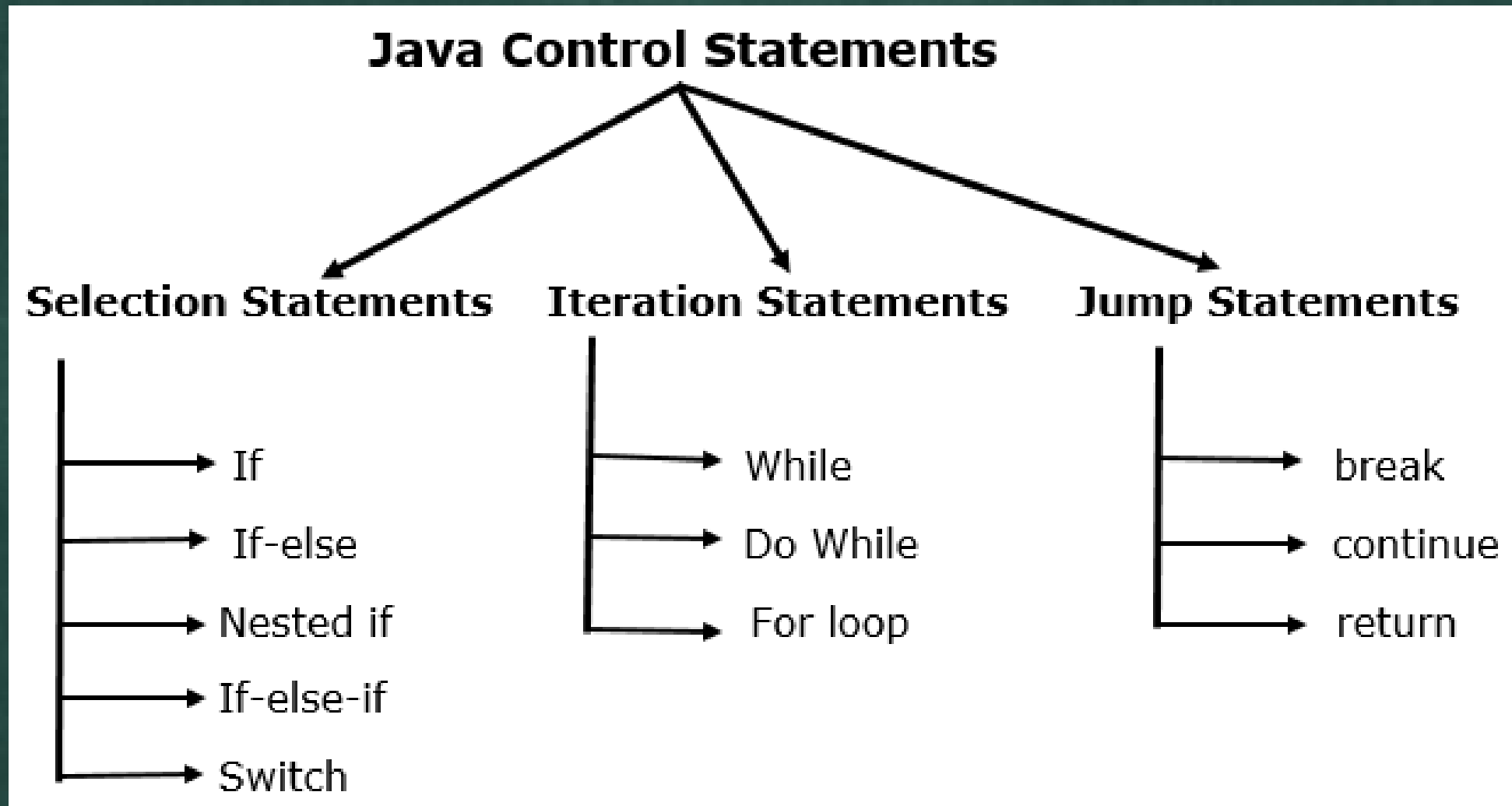
Control Statements in Java

Lecture 7

Control Statements

- The Java control statements inside a program are usually executed sequentially.
- Sometimes a programmer wants to break the normal flow and jump to another statement or execute a set of statements repeatedly.
- Control statements in java enables decision making, looping and branching.
- A control statement in java is a statement that determines whether the other statements will be executed or not.
- It controls the flow of a program.
- Control Statements can be divided into three categories, namely
 - Selection statements
 - Iteration statements
 - Jump statements

Classification Control Statements



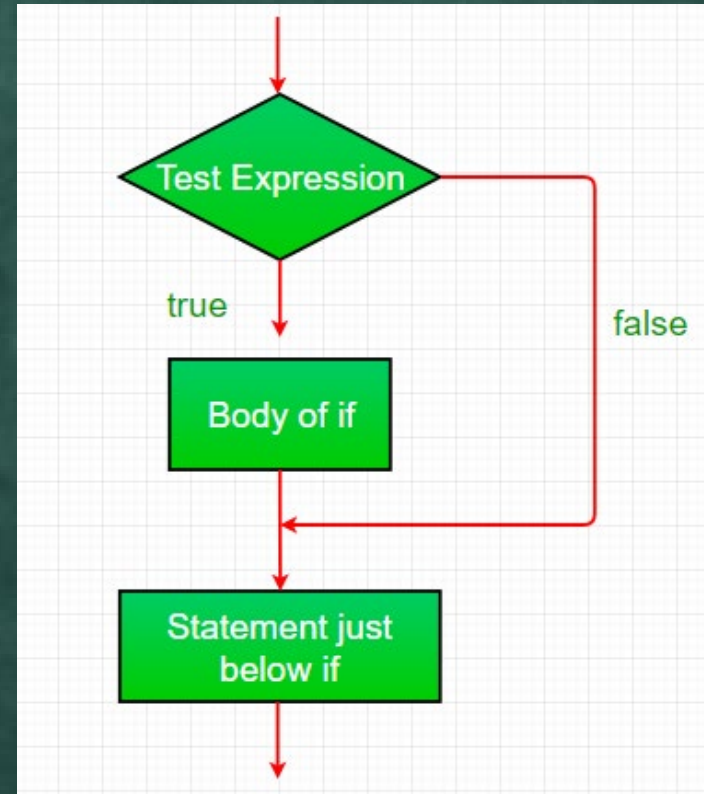
Decision-Making Statements

- Conditional Control Statements allows the program to select between the alternatives during the program execution.
- They are also called as decision-making statements or selection statements.
- Statements that determine which statement to execute and when are known as decision-making statements.
- These statements decides the flow of the execution based on some conditions.
- Java's Selection statements:
 - if
 - if-else
 - nested-if
 - if-else-if
 - switch-case

Simple if statement

- if statement is the simplest decision-making statement
- The if statement determines whether a code should be executed based on the specified condition.
- if statement's condition should evaluate only to the boolean values 'true' or 'false'.
- We can add more than one condition in the same if statement by using && or || operators.

```
if(condition) {  
    // statements (if Block)  
}  
//other statements
```



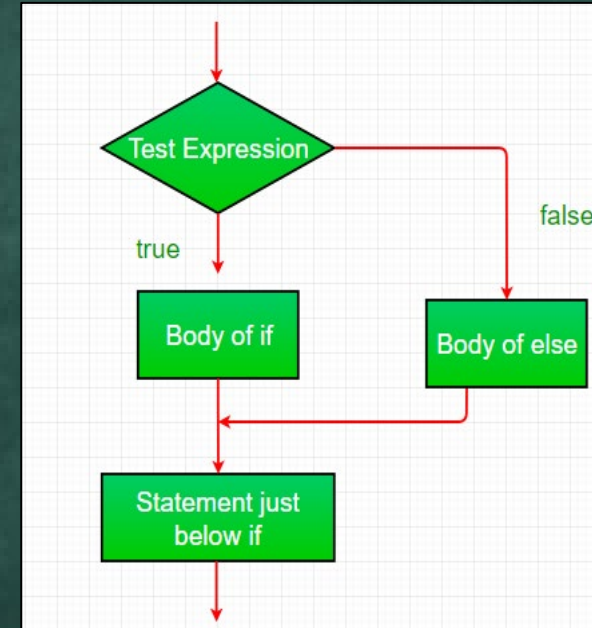
Example of if

```
class IfDemo1 {  
    public static void main(String[] args)  
    {  
        int marks=70;  
        if(marks > 65)  
        {  
            System.out.print("Hello Java! Am in If");  
        }  
    }  
}
```

if-then -else Statement

- The if-else statement is used for testing conditions.
- It is used for true as well as for false condition.
- If then Else statement provides two paths.
- The if block is executed when the condition holds true. When the condition evaluates to false, the statements inside the else block are executed.

```
if (condition){  
    // Executes this block if  
    // condition is true  
} else {  
    // Executes this block if  
    // condition is false  
}
```



Example of if-else statement

// Java program to illustrate if-else statement

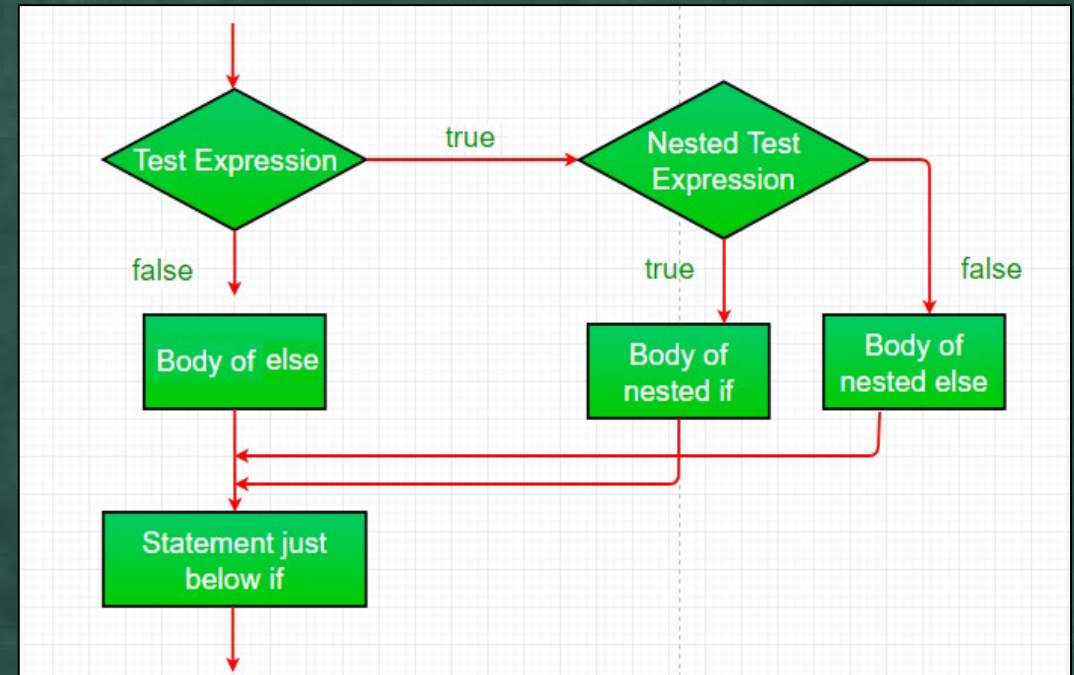
```
class IfElseDemo {  
    public static void main(String args[])  
    {  
        int i = 20;  
        if (i < 45)  
            System.out.println("i is smaller than 45");  
        else  
            System.out.println("i is greater than 55");  
    }  
}
```

Nested if statement

An if present inside an if block is known as a nested if block.

It is similar to an if..else statement, except they are defined inside another if..else statement.

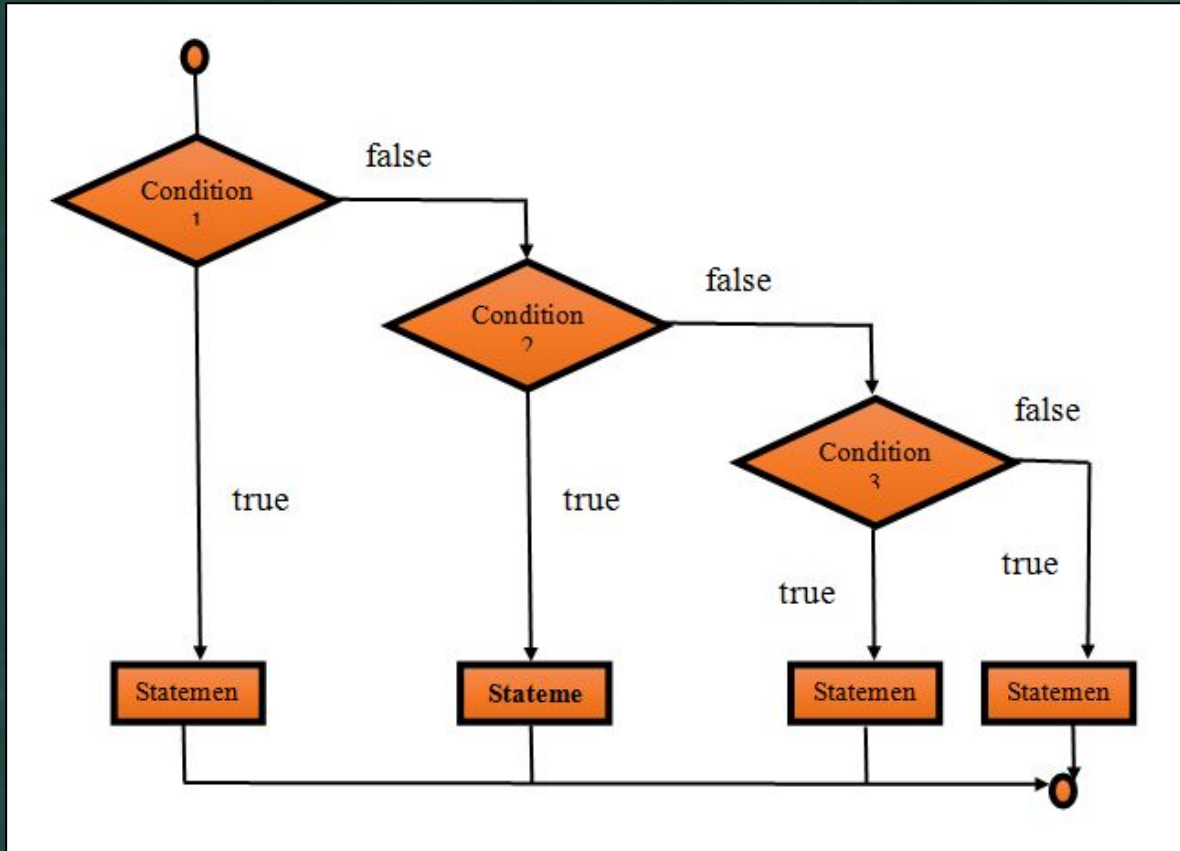
```
if (condition1)
{
    // Executes when condition1 is true
    if (condition2)
    {
        // Executes when condition2 is true
    }
}
```



Nested if example

```
class NestedIfDemo {  
    public static void main(String args[]) {  
        int s = 15;  
        if (s > 30){  
            if (s%2==0)  
                System.out.println("s is an even number and greater than 30!");  
            else  
                System.out.println("s is a odd number and greater than 30!");  
        } else {  
            System.out.println("s is less than 15");  
        }  
        System.out.println("Hello World!");  
    }  
}
```


If Else-If ladder:



```
class IfElseIfDemo{
public static void main(String []args){
int a=2; int b=3; int c=4;
if(a>b){
System.out.print("Institute1");
} else if(a>c){
System.out.print("Institute2");
} else {
System.out.print("Intellipaat");
}
}}
```


Switch Statement

- The switch statement is a multiway branch statement.
- A switch statement is used to execute a single statement from multiple conditions.
- The switch statement can be used with short, byte, int, long, enum types, etc.
- One or N number of case values can be specified for a switch expression.
- Duplicate case values are not allowed. A compile-time error is generated by the compiler if unique values are not used.
- The case value must be literal or constant. Variables are not permissible.
- Usage of break statement is made to terminate the statement sequence. It is optional.. If this statement is not specified, the next case is executed.
- Based on the argument in the switch statement suitable case value will be selected and executed.
- If no matching case found, then the default will be executed.

```
Switch(variable/value/expression){
```

```
Case :
```

```
// statements [];
```

```
Case :
```

```
// statements [];
```

```
...
```

```
default:
```

```
// statements [];
```

```
}
```

Example of Switch Statement

```
class Music {  
public static void main(String[] args) {  
int inst= 2;  
String musicInstrument;  
// switch statement with int data type  
switch (inst) {  
case 1:  
musicInstrument = "Guitar";  
break;  
case 2:  
musicInstrument = "Piano";  
break;  
case 3:  
musicInstrument = "Drums";  
break;  
default:  
musicInstrument = "Invalid";  
break;  
}  
System.out.println(musicInstrument);  
} }  
}
```

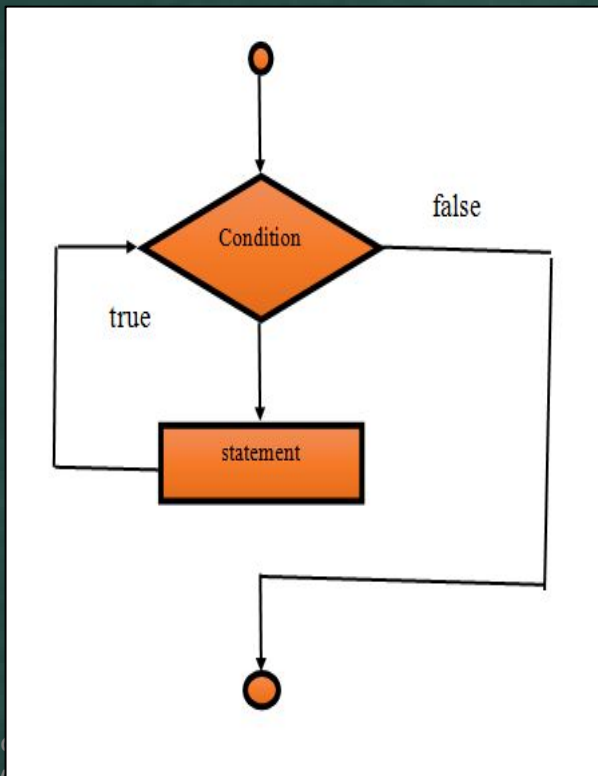
Looping Statements / Iterative Statements

- These are used to execute a block of statements multiple times.
- It means it executes the same code multiple times until a specified condition is met,
- These are also called Iteration statements.
- There are three types of looping control statements:

1. For loop
2. While loop
3. Do-while loop

While loop

```
while (test_expression)
{
    // statements
    update_expression;
}
```



// Program to display numbers 1 to 15 using do-while loop

```
class DoWhileDemo {
public static void main(String[] args)
{
    int i=1;
    do {
        System.out.println(i);
        i++;
    }while(i<=10);
} }
```

for Loop

For loop:

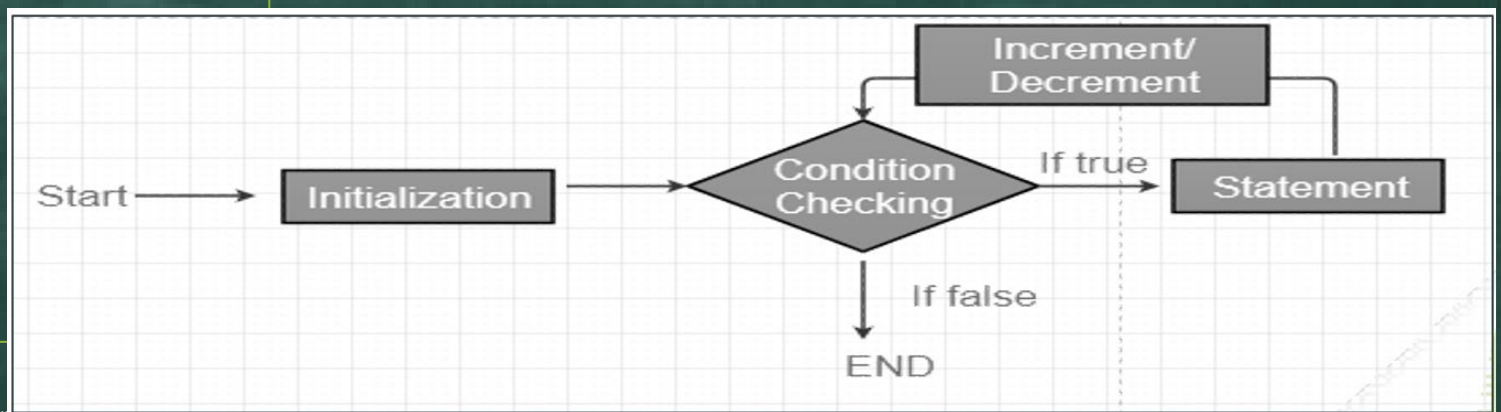
- The for loop in java is used to iterate and evaluate a code multiple times.
- When the number of iterations is known by the user, it is recommended to use the for loop.
- In java there are 3 types of for loops, they are as follows:

1. Simple for loop

2. For-each loop

3. labelled for loop

```
for      (initialization;      condition;  
increment/decrement)  
{  
    statement;  
}
```



Example of for loop

```
// Java program to illustrate for loop
class forLoopDemo {
    public static void main(String args[])
    {
        // Writing a for loop
        // to print Hello World 5 times
        for (int i = 1; i <= 5; i++)
            System.out.println("Hello
World");
    } }
```

- Output:
- Hello World
- Hello World
- Hello World
- Hello World
- Hello World

For-Each loop:

- The traversal of elements in an array can be done by the for-each loop.
- The elements present in the array are returned one by one.
- It must be noted that the user does not have to increment the value in the for-each loop.

```
public class ForEachDemo1
{
    public static void main(String[] args)
    {
        inta[]={100,101,102,103,104};
        for(int i:a)
        {
            System.out.println(i);
        }
    }
}
```

Output:

100
101
102
103
104

Labelled For Loop

```
labelname:  
for(initialization;condition;incr/decr)  
{  
    //code for execution  
}
```

```
public class LabeledForDemo {  
    public static void main(String[] args) {  
        num:  
            for(int i=1;i<=5;i++)  
            {  
                num1:  
                for(int j=1;j<=5;j++)  
                {  
                    if(i==2&& j==2)  
                    {  
                        break num;  
                    }  
                    System.out.println(i+" "+j);  
                }  
            }  
        }  
    }  
}
```

Output:

```
1 1  
1 2  
1 3  
1 4  
1 5  
2 1
```

Unconditional Control Statements / Jump Statements / Branching statements

- Branching statements are used to jump from the current executing loop.
- Branching statements in java are used to jump from a statement to another statement, thereby transferring the flow of execution

break Statement

- break is a keyword.
- It is used within any control statements.
- break - Terminates the loop or switch statement and transfers execution to the statement immediately following the loop or switch.
- The break statement in java is used to terminate a loop and break the current flow of the program.
- Syntax:

break; or
break <label>;

Using break statement in for loop

// Using break statement in for loop

```
class breakDemo
```

```
{
```

```
    public static void main(String args[])    {
```

```
        for (int j = 0; j <=10; j++)
```

```
        {
```

```
            // come out of loop when i is 5.
```

```
            if (j == 5)
```

```
                break;
```

```
            System.out.println(j);
```

```
        }
```

```
        System.out.println("After loop");
```

```
    }
```

```
}
```

Output:

0

1

2

3

4

5

After loop

Continue Statement

- continue is a keyword.
- In Java, the Continue statement is used in loops.
- Continue statement is used to jump to the next iteration of the loop immediately.
- It is used with for loop, while loop and do-while loop.

Syntax:

continue; or
continue<label>;

```
class continueTest {  
    public static void main(String args[])  
    {  
        for (int j = 0; j < 10; j++)  
        {  
            // If the number is odd then bypass and  
            // continue with next value  
            if (j%2 != 0)  
                continue;  
            // only even numbers will be printed  
            System.out.print(j + " ");  
        }  
    }  
}
```

Output: 0 2 4 6 8

continue with label

//continue with label is used to continue the number of loops below the label

```
class contlabelDemo{
public static void main(String args[]){
contLabel:
for(int i=1;i<=4;i++){
for(int k=1;k<=4;k++) {
System.out.print(" "+i+" ");
if(i==3)
    continue contLabel;
System.out.println("Hello");
}}}
```

return

- The last control statement is return.

```
// Demonstrate return.  
class Return {  
    public static void main(String args[]) {  
        boolean t = true;  
  
        System.out.println("Before the return.");  
  
        if(t) return; // return to caller  
  
        System.out.println("This won't execute.");  
    }  
}
```

The output from this program is shown here: **Before the return.**

Thank You

08-Nov-25